

Binary Search Tree

What is it ?

- A binary tree
- All left subtrees's node value $\leq$ parent node value
- All right subtrees's node value $>$ parent node value
- More like Quicksort

Find ?

- compare the current node's value, then recursively search left if the target is smaller, or right if the target is larger, until the value is found or a null node is reached.
- pseudocode:
 - null node $\rightarrow$ nothing there $\rightarrow$ return false
 - node.value = target $\rightarrow$ it's here $\rightarrow$ return true
 - node.value < target $\rightarrow$ target must be on the right $\rightarrow$ go to the right
 - node.value > target $\rightarrow$ target must be on the left $\rightarrow$ go to the left
- $O(h)$: h-height of tree (which range from $\log n$ - n)

Insert

- comparing the value to the current node: if less than or equal, go left; if greater, go right, until a null position is found
- pseudocode:

```
if (node == null) {
    return { value: v }
} if (node.v < v) {
    node.right = insert(node.r, v)
} else {
    node.left = insert(node.l, v)
}
```

Delete

- Case 1: no child, just delete
- Case 2: 1 child, set parent connection to child (of deleted node)
- Case 3: 2 child

- Find smallest node of larger subtree || Find largest node of smaller subtree
- Replace deleted node with this node
- This node can only have 0 or 1 child
 - If 0 child, just replace
 - If 1 child, replace and point this node parent to its child
- In case 3, when having 2 directions to go, should choose the replacement node from the subtree with larger height can help maintain a more balanced tree, potentially keeping the tree's structure more compact and improving future search performance.

Q&A

Q: What are some practical use cases for Binary Search Trees?

A: BSTs excel at searching lists of data (e.g. arrays/maps/sets) where the data is changing frequently (adding and removed). For example, file systems (quickly locate files and directories) or text autocomplete (efficiently store and search through large dictionaries of words).

Q: Some balance tree methods ?

A: AVL (rotate recursively) and Red-black tree. if finding elements is more common, AVL might be better; if insertions are more frequent, Red-Black trees could be preferable.

Fact: In-order traverse in binary search tree leads to ordered list